

Expt 7	Spectral Estimation
Date:	7.1 Power Spectral Density Estimation and Spectral Analysis of Noisy Sinusoidal Signals Using Periodogram Method in MATLAB

Code:

```

clc;
clear;
close all;

%% Signal Generation
fs = 1000;           % Sampling frequency (Hz)
t = (0:999)/fs;      % Time vector

% Original signal (100 Hz + 200 Hz)
s = sin(2*pi*100*t) + 0.5*sin(2*pi*200*t);

% Add Gaussian White Noise
n = 0.3*randn(size(t));

% Noisy signal
x = s + n;

%% Figure 1: Noisy Signal
figure;
plot(t, x, 'LineWidth',1.2);
grid on;
xlabel('Time (s)');
ylabel('Amplitude');
title('Signal with Noise');

%% Figure 2: Power Spectral Density (PSD)
figure;
periodogram(x, [], [], fs);
title('Power Spectral Density using Periodogram');

%% Figure 3: Magnitude Spectrum using FFT
X = abs(fft(x));
f = (0:length(X)-1)*fs/length(X);

figure;
plot(f(1:500), X(1:500), 'LineWidth',1.5);
grid on;
xlabel('Frequency (Hz)');
ylabel('Magnitude');
title('Magnitude Spectrum');
xlim([0 fs/2]);

%% Signal and Noise Power

```

```

signalPower = mean(s.^2);
noisePower = var(n);

fprintf('Signal Power = %.4f\n', signalPower);
fprintf('Noise Power = %.4f\n', noisePower);

```

%% Figure 4: Signal vs Noise Power

```

figure;
bar([signalPower noisePower]);
set(gca,'XTickLabel',{'Signal','Noise'});
ylabel('Power');
title('Signal and Noise Power Comparison');
grid on;

```

Output:

Fig 1:

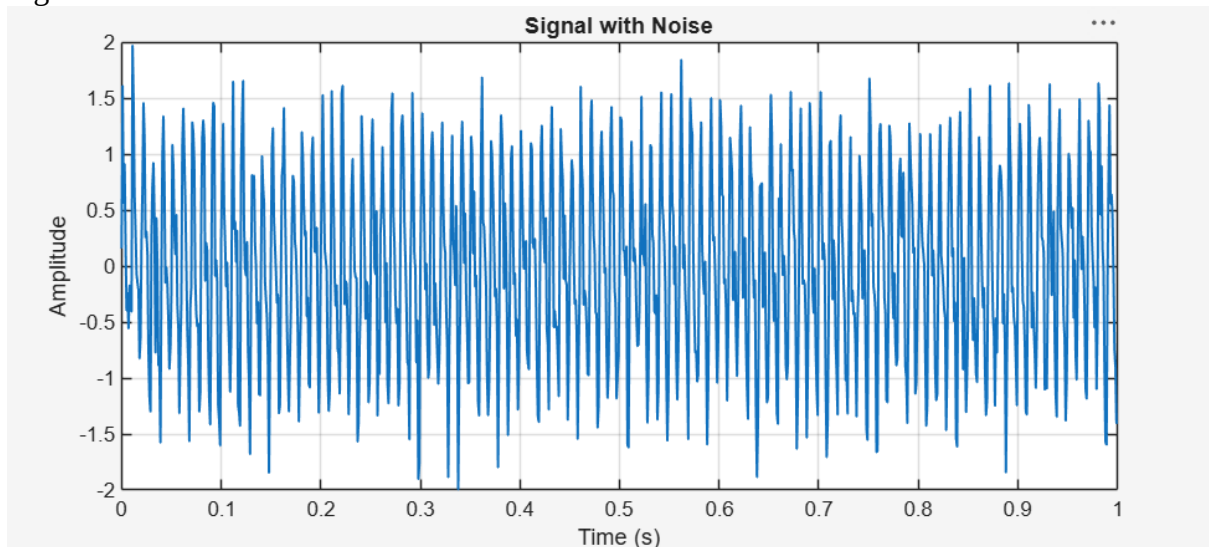


Fig 2:

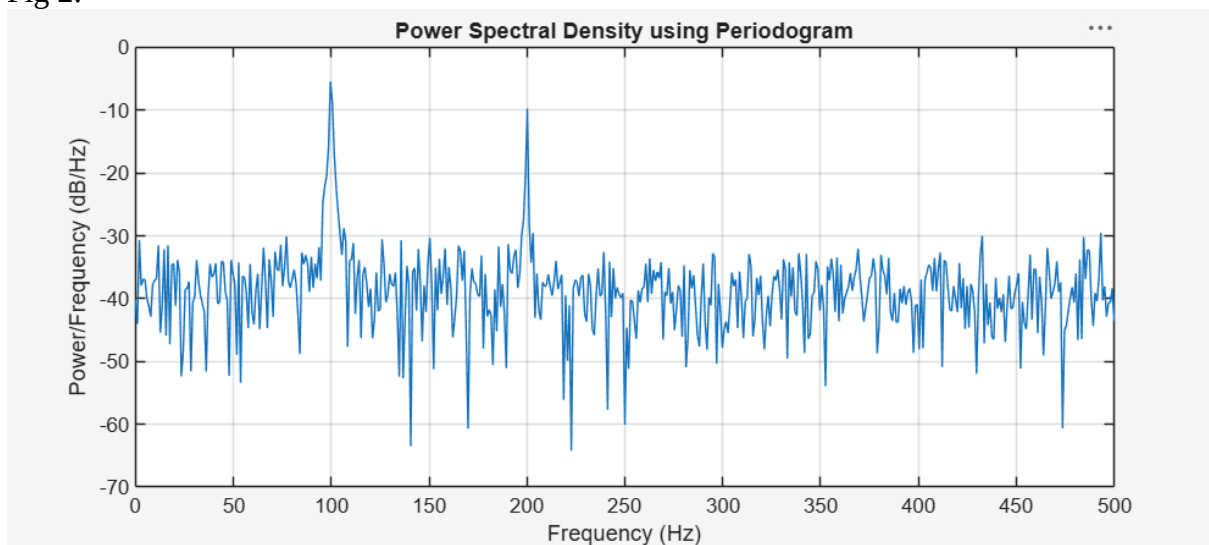


Fig 3:

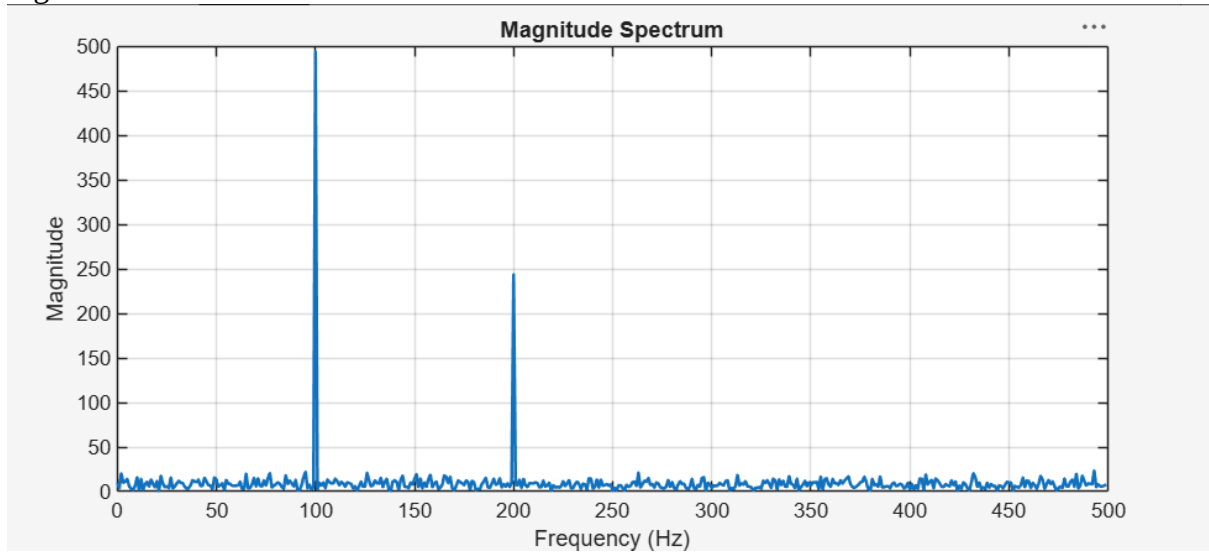
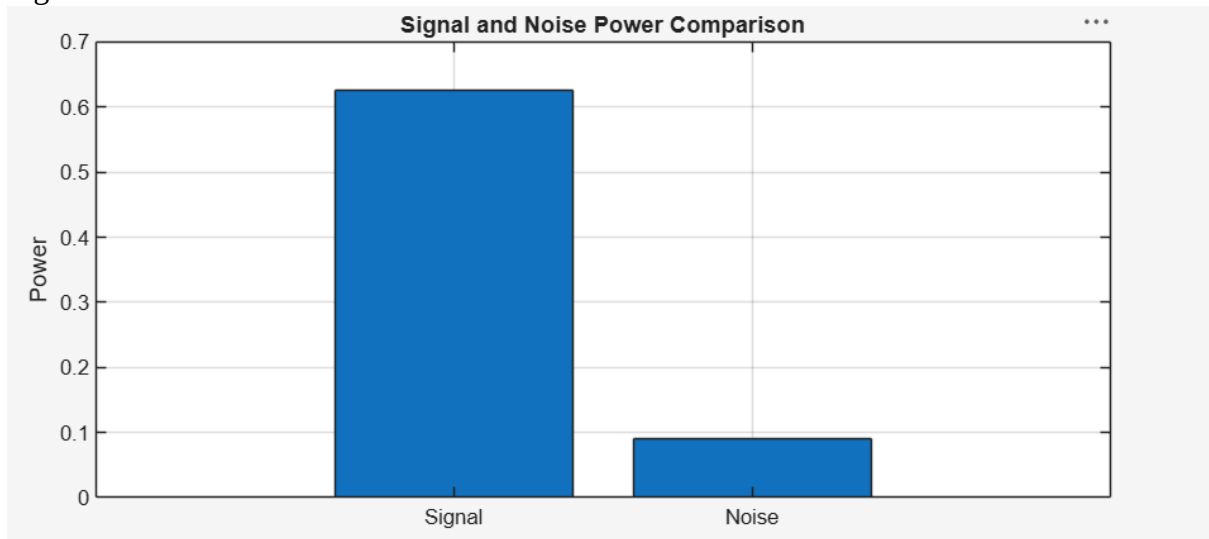


Fig 4:



Expt 7	Spectral Estimation
Date:	7.2 Analysis of Windowing Effects on Spectral Estimation Using Rectangular, Hamming, and Hanning Windows in MATLAB

Code:

clc;

clear;

close all;

%% Signal Generation

fs = 1000; % Sampling frequency (Hz)

N = 256; % Number of samples

t = (0:N-1)/fs; % Time vector

% Two closely spaced frequency components

x = sin(2*pi*100*t) + 0.8*sin(2*pi*110*t);

%% FFT Parameters

NFFT = 2048; % FFT length (Zero Padding)

f = (0:NFFT/2-1)*(fs/NFFT);

%% Window Functions

w = {rectwin(N), hamming(N), hann(N)}; % Use hanning(N) if required

names = {'Rectangular', 'Hamming', 'Hann'};

%% Apply Windows and Compute FFT

for i = 1:3

 % Apply window

 xw = x(:).*w{i};

 % FFT

 X = fft(xw,NFFT);

 % Magnitude Spectrum

 P{i} = abs(X(1:NFFT/2));

 % Normalize

 P{i} = P{i}/max(P{i});

end

%% Individual Spectra

figure;

for i = 1:3

```

subplot(3,1,i);
plot(f,20*log10(P{i}+eps),'LineWidth',1.5);

grid on;
xlim([0 250]);

xlabel('Frequency (Hz)');
ylabel('Magnitude (dB)');
title(['Spectrum using ' names{i} ' Window']);

```

end

%% Comparison of Windows

```

figure;

plot(f,20*log10(P{1}+eps),'k','LineWidth',1.5);
hold on;
plot(f,20*log10(P{2}+eps),'r','LineWidth',1.5);
plot(f,20*log10(P{3}+eps),'b','LineWidth',1.5);

grid on;
xlim([50 170]);

xlabel('Frequency (Hz)');
ylabel('Magnitude (dB)');
title('Comparison of Window Functions');
legend(names);

```

%% Window Shapes

```

figure;

for i = 1:3

    subplot(3,1,i);

    plot(w{i},'LineWidth',1.5);
    grid on;

    xlabel('Sample Number');
    ylabel('Amplitude');
    title([names{i} ' Window']);

```

end

%% Peak Detection

```

for i = 1:3

    [pks{i},loc{i}] = findpeaks(P{i},f,'MinPeakHeight',0.3);

    fprintf('\n%s Window Detected Frequencies:\n', names{i});

```

```

    disp(loc{i});

end

%% Peak Marking (Hamming Window)
figure;

plot(f,20*log10(P{2}+eps),'LineWidth',1.5);
hold on;

plot(loc{2},20*log10(pks{2}+eps),...
     'ro','MarkerFaceColor','r');

grid on;
xlim([50 170]);

xlabel('Frequency (Hz)');
ylabel('Magnitude (dB)');
title('Detected Frequencies (Hamming Window)');

%% Main Spectral Peaks
fprintf('\nMain Spectral Peaks\n');
fprintf('-----\n');

for i = 1:3

    [~,idx] = max(P{i});
    fprintf('%-12s Window : %.2f Hz\n', names{i}, f(idx));

end

```

Output:

Fig 1:

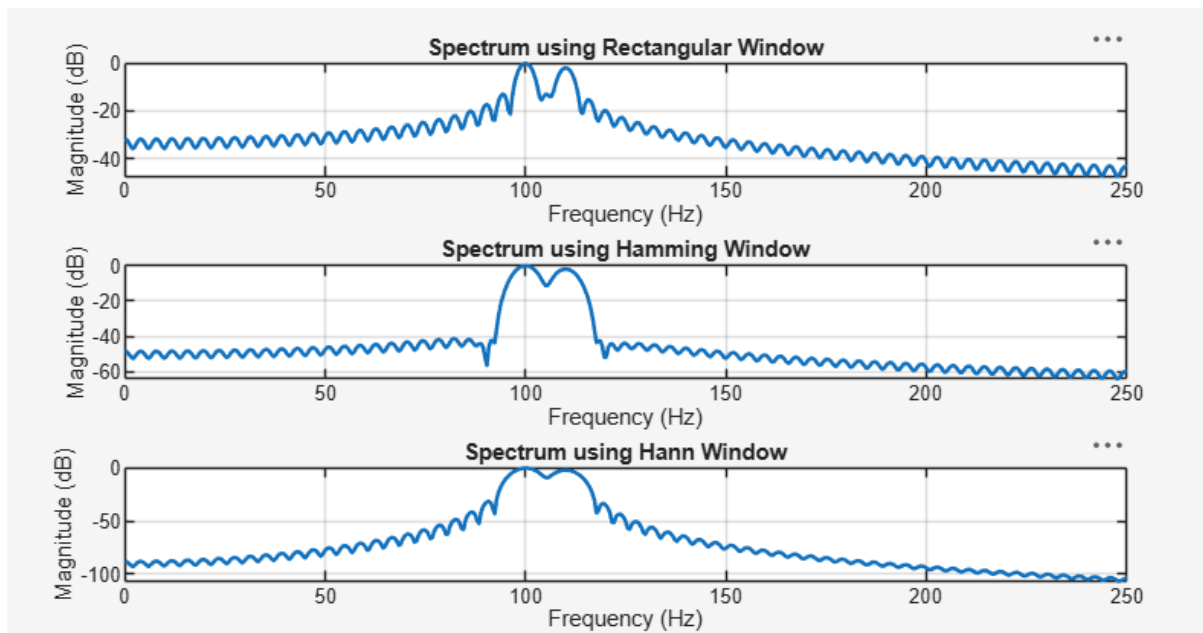


Fig 2:

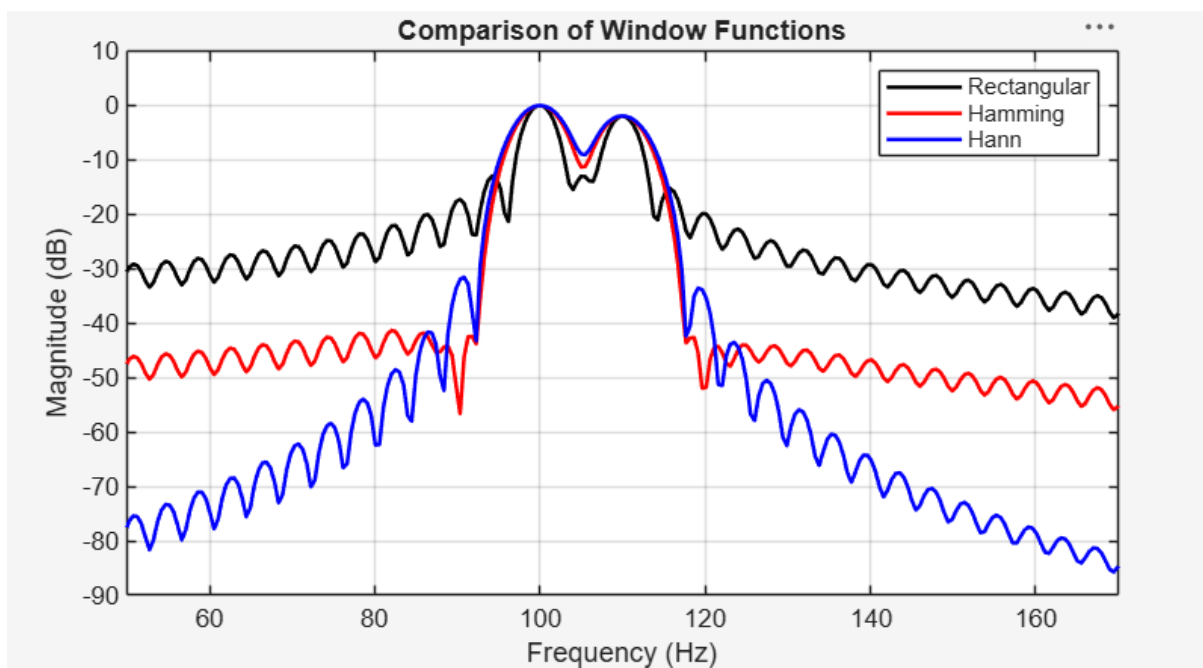


Fig 3:

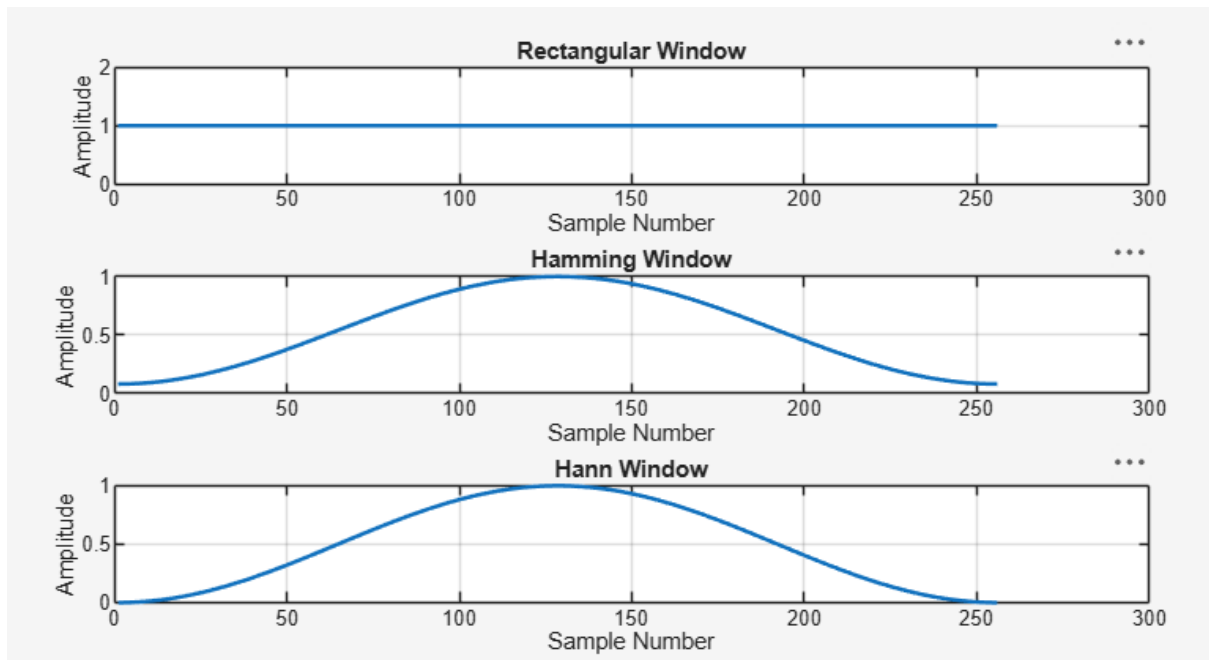


Fig 4:

